

Barakova Luxury Travel — Технически учебник за проекта

Версия: 1.0

Дата: Юли 2026

Автор на проекта: Bobby Uzunov

Live сайт: <https://barakovaluxurytravel.com>

GitHub: <https://github.com/BobbyUzunov/barakova-luxury-travel>

Съдържание (Table of Contents)

1. [Въведение](#)
2. [Стек](#)
3. [Структура на проекта](#)
4. [Routing](#)
5. [Components](#)
6. [Styling](#)
7. [SEO](#)
8. [Контактна форма](#)
9. [Security](#)
10. [Performance](#)
11. [Deployment](#)
12. [Git](#)

13. [Реални примери](#)

14. [Какво научих](#)

15. [Какво да уча след това](#)

- [Индекс](#)

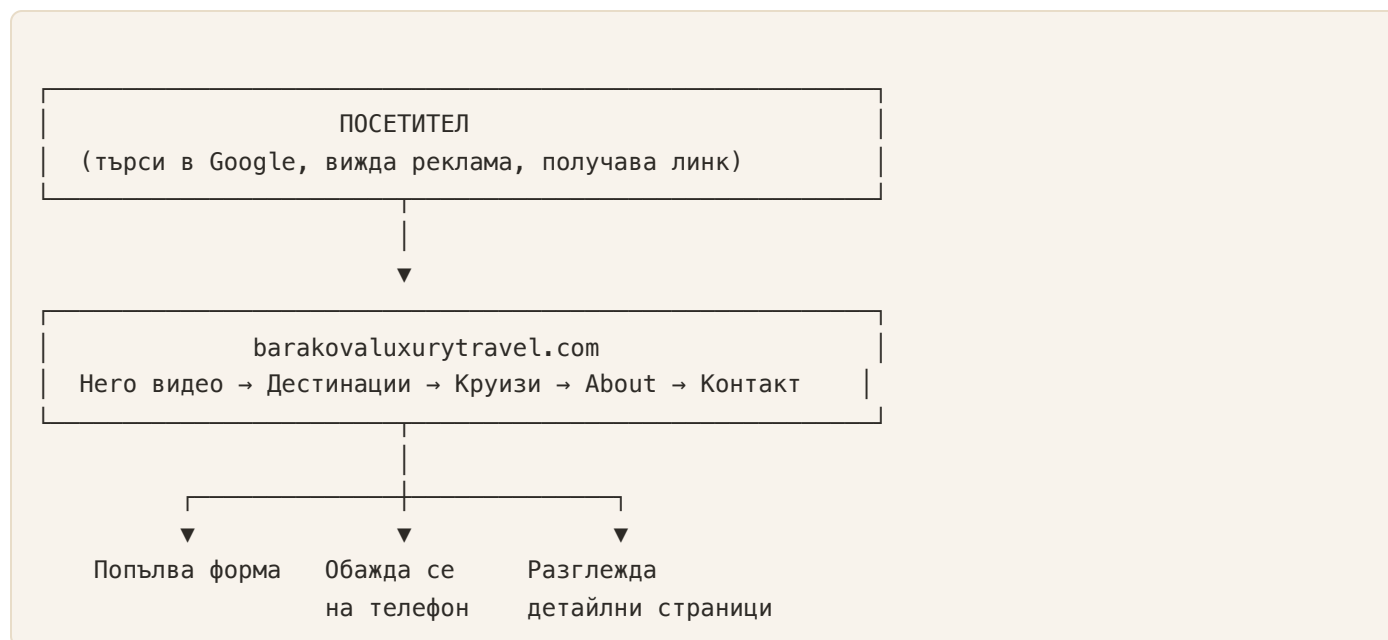
1. Въведение

1.1 Каква е идеята на проекта?

Barakova Luxury Travel е уебсайт за луксозни пътувания. Собственикът е **Богдана Баракова** — консултант, който помага на клиенти да изберат дестинации, круизи и бутикови хотели.

Сайтът не е онлайн магазин. Той е **digital витрина + контактна точка**:

- Показва услуги, дестинации и круизи
- Изгражда доверие (about секция, снимки, блог)
- Събира запитвания чрез форма и телефон
- Работи на **български** и **английски**



1.2 Какви проблеми решава?

Проблем	Решение в проекта
Клиентът няма онлайн присъствие	Професионален сайт с домейн
Трудно се намира в Google	SEO: metadata, sitemap, structured data
Запитвания по телефон/имейл са хаотични	Контактна форма с валидация и имейл известия
Спам ботове	Turnstile CAPTCHA + rate limiting + honeypot
Чуждестранни клиенти	Двуетичност /bg и /en
Липса на доверие	Premium гуизайн, Vugeo hero, testimonials
GDPR изисквания	Cookie banner + privacy policy

1.3 Защо е избрана тази архитектура?

Проектът използва **Next.js App Router** — модерен подход за React приложения.

Защо Next.js, а не просто HTML/CSS?

- Един проект → много страници (начало + 12 дестинации + 6 круиза + 3 блога × 2 езика)
- SEO изисква server-rendered HTML (Google да вижда съдържанието)
- API route за контакт формата (без отделен backend сървър)
- Статично генериране (SSG) → бърз сайт

Защо не WordPress?

- Пълен контрол над кода и performance
- По-лесно custom гуизайн
- TypeScript хваща грешки преди deploy
- Vercel deploy с един `git push`

Защо съдържанието е в `constants/`, а не в CMS?

- За този проект съдържанието се променя рядко
- По-просто за поддръжка от един разработчик

- CMS (Sanity) е планирано за бъдеща фаза

2. Стек

Тук обясняваме всяка технология, която проектът използва.

2.1 Next.js 16

Какво е?

Framework върху React. Добавя routing, server rendering, API routes, оптимизация на изображения.

Защо го ползваме?

Един инструмент за frontend + backend + deploy. В проекта: `next@^16.2.9`.

Алтернативи: Remix, Gatsby, Astro, чист React + Vite.

Предимства		Недостатъци	
Отлично SEO (SSG/SSR)		Крива на обучение	
File-based routing	Промените между версии понякога чунят ког		
Vercel integration	По-тежък от чист Vite		
next/image оптимизация			

2.2 React 19

Какво е?

JavaScript библиотека за изграждане на UI чрез **компоненти**.

Защо го ползваме?

Next.js е построен върху React. Компонентите се преизползват (header, footer, modal).

Алтернативи: Vue, Svelte, Angular.

Предимства	Недостатъци
Огромна общност	JSX синтаксис — нов за начинаещи
Компонентен модел	Трябва да разбереш state и effects
Огромна екосистема	

2.3 TypeScript

Какво е?

JavaScript + **типове**. Ако подгнеш грешен тип, компилаторът казва грешката преди да пуснеш сайта.

Пример от проекта:

```
// constants/i18n.ts
export function isLocale(value: string): value is Locale {
  return value === "bg" || value === "en";
}
```

Тук TypeScript знае: ако `isLocale(x)` е `true`, то `x` е `"bg" | "en"`.

Алтернативи: Чист JavaScript.

Предимства	Недостатъци
По-малко runtime грешки	Повече код за писане
По-добър autocomplete в IDE	По-бавно обучение в началото

2.4 App Router

Какво е?

Новият routing модел на Next.js. Пакета `app/` определя URL-ите.

Защо App Router, а не Pages Router?

- По-добър layout system (вложени layouts)
- Server Components по подразбиране

- Parallel routes (@modal) за модали
- generateMetadata за SEO

Алтернативи: Pages Router (старият Next.js), React Router.

2.5 Server Components vs Client Components

Server Component — рендерира се на сървъра. Няма "use client". Не може useState , onClick .

Client Component — рендерира се и на сървъра (първоначално), после React поема в браузъра. Има "use client" в началото на файла.

Пример от проекта:

```
app/[locale]/page.tsx          → Server Component (няма "use client")
app/components/home/home-page.tsx → Client Component ("use client")
```

Защо разделението?

- Server: по-малко JavaScript към браузъра
- Client: интерактивност (форма, меню, модали, видео)

2.6 Tailwind CSS 4

Какво е?

Utility-first CSS framework. Вместо .my-button { padding: 1rem } пишеш className="px-4 py-2".

В проекта: Tailwind + голям custom CSS файл (globals.css).

```
// Пример от home-page.tsx
<section className="section-shell px-5 sm:px-8 lg:px-12">
```

- px-5 — padding хоризонтално
- sm:px-8 — от 640px нагоре → повече padding
- lg:px-12 — от 1024px нагоре → още повече

Алтернативи: Bootstrap, plain CSS, CSS Modules, styled-components.

Предимства	Недостатъци
Бързо layout-ване	Дълги className низове
Responsive с префикси	Трудно четене без опит
Малък production CSS (purge)	

2.7 Vercel

Какво е?

Cloud платформа за хостване на Next.js. Свързва се с GitHub — всеки push deploy-ва сайта.

Защо Vercel?

Създадена от екипа на Next.js. Zero-config deploy.

Алтернативи: Netlify, AWS, DigitalOcean, self-hosted.

Предимства	Недостатъци
Автоматичен deploy от Git	Vendor lock-in (частично)
SSL, CDN, edge	Платен при голям трафик
Environment variables UI	

2.8 Resend

Какво е?

API услуга за изпращане на имейли от приложения.

Защо го ползваме?

Контактната форма трябва да изпрати имейл до Богдана. Resend е прост — едн `fetch` POST заявка.

```
// app/api/contact/route.ts (опростено)
await fetch("https://api.resend.com/emails", {
  method: "POST",
  headers: { Authorization: `Bearer ${resendApiKey}` },
  body: JSON.stringify({ from, to, subject, html }),
});
```

Алтернативи: SendGrid, Mailgun, Amazon SES, Nodemailer + SMTP.

Предимства	Недостатъци
Лесна интеграция	Платен слег free tier

2.9 ImprovMX

Какво е?

Email forwarding услуга. Пренасочва `info@barakovaluxurytravel.com` → Gmail inbox.

Защо го ползваме?

Клиентът иска професионален имейл адрес, но чете пощата си в Gmail.

```
Посетител вижда: info@barakovaluxurytravel.com
      |
      ▼
    ImprovMX (DNS MX records)
      |
      ▼
    Gmail inbox на клиента
```

Resend vs ImprovMX:

- **Resend** — изпраща имейли **ОТ** сайта (контакт форма)
- **ImprovMX** — пренасочва входящи имейли **КЪМ** Gmail

Алтернативи: Google Workspace, Zoho Mail, Cloudflare Email Routing.

2.10 Vimeo

Какво е?

Vimeo хостинг платформа.

Защо го ползваме?

Hero секцията има cinematic background video. Vimeo позволява embed с autoplay, muted, loop.

```
// constants/hero-video.ts
export const heroVimeoVideoId = "1204531589";
```

Алтернативи: YouTube embed, self-hosted video, Cloudinary.

Добра deliverability	Изисква domain verification
Предимства	Недостатъци
Качествено streaming	Зависимост от външен CDN
Background mode	CSP трябва да позволи vimeo.com
По-професионален bug от YouTube	

2.11 Cloudflare Turnstile

Какво е?

CAPTCHA алтернатива — проверява дали потребителят е човек, не бот.

Защо го ползваме?

Публична контакт форма без CAPTCHA = спам.

```
// app/components/turnstile-widget.tsx
<Turnstile siteKey={siteKey} onSuccess={onTokenChange} />
```

Алтернативи: Google reCAPTCHA, hCaptcha, Vercel BotID.

Предимства	Недостатъци
Безплатен	Още една външна зависимост
По-малко гразнец от reCAPTCHA	Трябват env keys

Privacy-friendly	
------------------	--

2.12 Structured Data (JSON-LD)

Какво е?

Машинно четим формат, който казва на Google **какво представлява** страницата (бизнес, човек, статия).

Пример от `app/layout.tsx` :

```
{
  "@type": "TravelAgency",
  "name": "Barakova Luxury Travel",
  "email": "info@barakovaluxurytravel.com",
  "telephone": "+359883770909"
}
```

Защо е важно?

Може да гледате rich results в Google (знания панел, контакти).

2.13 SEO (обобщение на технологиите)

SEO в проекта не е една библиотека — комбинация от:

- `metadata` в layouts и pages
- `sitemap.xml` и `robots.txt`
- Open Graph / Twitter Cards
- hreflang за BG/EN
- JSON-LD structured data
- Semantic HTML (`<header>`, `<main>`, `<article>`)
- Бързо зареждане (SSG, image optimization)

(Подробно в секция 7.)

3. Структура на проекта

3.1 Пълно дърво

```

Barakova Travel/
├── app/                                # Next.js App Router – всички страници и API
│   ├── [locale]/                      # Динамичен сегмент: /bg или /en
│   │   ├── @modal/                   # Parallel route – модали без напускане на home
│   │   │   ├── (.)destinations/[slug]/page.tsx
│   │   │   ├── (.)cruises/[slug]/page.tsx
│   │   │   ├── (.)blog/[slug]/page.tsx
│   │   │   └── default.tsx
│   │   ├── destinations/[slug]/page.tsx
│   │   ├── cruises/[slug]/page.tsx
│   │   ├── blog/[slug]/page.tsx
│   │   ├── privacy/page.tsx
│   │   ├── layout.tsx                # Layout за всеки език + modal slot
│   │   ├── loading.tsx
│   │   └── page.tsx                  # Начална страница /bg или /en
│   ├── api/
│   │   └── contact/route.ts          # POST endpoint за формата
│   ├── components/                  # React компоненти
│   │   ├── home/                    # Компоненти само за началната страница
│   │   ├── locale-html.tsx
│   │   └── turnstile-widget.tsx
│   ├── analytics.tsx                # Google Analytics (след cookie consent)
│   ├── content-detail-page.tsx       # Шаблон за детайлни SEO страници
│   ├── cookie-consent.tsx           # GDPR банер
│   ├── globals.css                  # Глобални стилове + Tailwind
│   ├── icon.svg                     # SVG икона (източник за PWA)
│   ├── layout.tsx                   # Root layout – HTML shell, JSON-LD
│   ├── manifest.ts                  # PWA manifest
│   ├── not-found.tsx                # 404 страница
│   ├── robots.ts                    # robots.txt
│   └── sitemap.ts                    # sitemap.xml
├── constants/                        # Статично съдържание и конфигурация
│   ├── content.ts                   # TypeScript типове за съдържание
│   ├── content-bg.ts                # Български текстове
│   ├── content-en.ts                # Английски текстове
│   ├── destination-images.ts        # Логика за снимки на дестинации
│   ├── detail-ui.ts                 # UI текстове за детайлни страници
│   ├── hero-video.ts                # Vimeo конфигурация
│   ├── i18n.ts                      # Езикови помощници
│   ├── images.ts                    # Пътища към hero/profile снимки
│   ├── locale-metadata.ts           # SEO title/description по език
│   └── privacy.ts                    # Privacy policy + cookie текстове

```

```
| ├── seo-content.ts           # Slug-ове + SEO данни
| ├── site.ts                 # URL, имейл, телефон
| docs/                       # Документация
| ├── client-handoff-checklist.md
| ├── barakova-learning-guide.md ← този файл
| lib/                         # Споделена бизнес логика (не UI)
| ├── contact.ts              # Валидация, HTML имейл
| ├── contact-api-messages.ts # Локализирани API грешки
| ├── contact.test.ts         # Unit тестове
| ├── rate-limit.ts           # Rate limiting
| ├── use-modal-accessibility.ts
| ├── verify-turnstile.ts      # Turnstile server verification
| public/                     # Статични файлове (директен URL)
| ├── hero-bogdana-beach.webp
| ├── images/
| | ├── barakova-1.webp
| | ├── barakova-2.webp
| | └── destinations/         # Бъдещи клиентски снимки
| ├── icon-192.png
| ├── icon-512.png
| └── icons/phone.svg
| scripts/                    # Build/helper скриптове
| ├── generate-icons.mjs
| └── optimize-images.mjs
| proxy.ts                    # Redirects (/ → /bg, legacy paths)
| next.config.ts              # Next.js конфигурация
| package.json                 # Зависимости и npm scripts
| tsconfig.json                # TypeScript настройки
| .env.example                 # Шаблон за environment variables
| README.md                    # Deploy инструкции
```

3.2 Обяснение на всяка панка

Папка	Предназначение
app/	Сърцето на Next.js. Всеки <code>page.tsx</code> = URL.
app/[locale]/	Всички страници са пог <code>/bg/...</code> или <code>/en/...</code>
app/api/	Backend endpoints. Само <code>contact</code> в този проект.
app/components/	React компоненти, разделени от routing логиката
constants/	Текстове и конфигурация — не е база данни, а TypeScript файлове

lib/	Чиста логика без UI — тества се лесно
public/	Файлове с фиксиран URL: /hero-bogdana-beach.webp
scripts/	Node скриптове за изображения и икони
docs/	Документация за хора

Правило: Ако нещо е UI → `app/components/`. Ако е логика → `lib/`. Ако е мексн → `constants/`.

4. Routing

4.1 Как работи App Router?

В Next.js **name** = URL.

```

app/[locale]/page.tsx           → /bg или /en
app/[locale]/privacy/page.tsx   → /bg/privacy
app/[locale]/destinations/[slug]/page.tsx → /bg/destinations/maldives

```

`[locale]` и `[slug]` са **динамични сегменти** — placeholder-u.

4.2 layout.tsx

Layout обвива страниците. Не се презарежда при навигация между child pages.

```

app/layout.tsx           ← <html>, <body>, cookie banner, analytics
├─ app/[locale]/layout.tsx ← locale validation, modal slot
│   └─ page.tsx             ← съдържание

```

Root layout (`app/layout.tsx`):

- Задава глобални metadata
- Вкарва JSON-LD structured data
- Рендерува `{children}` + `CookieConsent` + `Analytics`

Locale layout (`app/[locale]/layout.tsx`):

- Проверява дали locale е валиден (bg или en)
- Рендерува {children} у {modal} (parallel route)

4.3 page.tsx

Всеки page.tsx е една страница. Може да е Server или Client Component.

```
// app/[locale]/page.tsx
export default async function Page({ params }: PageProps) {
  const { locale } = await params;
  if (!isLocale(locale)) notFound();
  return <HomePage locale={locale} />;
}
```

Рег по рег:

1. params ухва от URL (/bg → locale = "bg")
2. Ако locale е невалиден → 404
3. Рендерува HomePage с правилния език

4.4 Intercepting Routes (модали с URL)

Проблем: Искаме при клик на дестинация да се отвори модал, но URL да стане /bg/destinations/maldives (за SEO и споделяне).

Решение: Parallel route @modal + intercepting (.)

```
app/[locale]/
├── page.tsx                # Home
├── destinations/[slug]/page.tsx # Пълна страница (директен линк)
└── @modal/(.)destinations/[slug]/page.tsx # Модал (от home)
```

(.) означава: „прихвани същия route, но покажи модал вместо пълна страница“.

Потребител на /bg кликва "Малдиви"

↓

URL става /bg/destinations/maldives

↓

@modal slot показва InterceptedContentModal
(overlay върху home, router.back() затваря)

4.5 Dynamic Routes + generateStaticParams

За SEO Next.js **предварително генерира** всички страници при build:

```
// app/[locale]/destinations/[slug]/page.tsx
export function generateStaticParams() {
  return getSeoDestinations("bg").flatMap((destination) => [
    { locale: "bg", slug: destination.slug },
    { locale: "en", slug: destination.slug },
  ]);
}
```

Резултат: 95 статични HTML страници при `npm run build`.

4.6 Metadata

Metadata се задава на три нива:

1. **Root** — `app/layout.tsx` → `export const metadata`
2. **Locale** — `app/[locale]/layout.tsx` → `generateMetadata()`
3. **Демайл** — `destinations/[slug]/page.tsx` → `generateMetadata()` с име на дестинацията

4.7 proxy.ts (redirects)

Файлът `proxy.ts` (Next.js 16 proxy/middleware) пренасочва:

```
/ → /bg
/privacy → /bg/privacy
/destinations/maldives → /bg/destinations/maldives (legacy)
```

5. Components

5.1 Защо има компоненти?

Без компоненти целият сайт би бил един файл с 3000+ реда. Компонентите разделят UI на **малки, преизползваеми части**.

```
HomePage
├── SiteHeader
├── HeroBackground
├── DestinationImage (× много)
├── ContactSection
│   └── TurnstileWidget
├── SiteFooter
└── Modals (ContentModal, BlogModal)
```

5.2 Как общувам?

Props — данни от родител към дете:

```
<ContactSection content={content} locale={locale} />
```

State — вътрешни данни на компонента:

```
const [isSubmitting, setIsSubmitting] = useState(false);
```

Callbacks — дете уведомява родител:

```
<HeroBackground onVideoActiveChange={setHeroVideoActive} />
```

Router — навигация:

```
const router = useRouter();
router.back(); // затваря модал
router.push(localizedHash(locale, "#contact")); // към контакт секция
```

5.3 Composition (съставяне)

Големи компоненти се съставят от малки:

- `InterceptedContentModal` → използва `ContentModal`
- `ContentModal` → използва `useModalAccessibility` + `next/image`
- `ContactSection` → използва `TurnstileWidget`

5.4 Server vs Client в компонентите

Компонент	Тип	Защо
<code>app/[locale]/page.tsx</code>	Server	Само подава locale
<code>home-page.tsx</code>	Client	State, scroll, меню
<code>contact-section.tsx</code>	Client	Form state, fetch
<code>hero-background.tsx</code>	Client	Video, useEffect
<code>destination-image.tsx</code>	Client	onError fallback

6. Styling

6.1 Хибриден подход

Проектът комбинира:

1. **Tailwind utilities** — spacing, grid, responsive padding
2. **Custom CSS classes** в `globals.css` — premium гизайн система

```
/* globals.css */
:root {
  --ivory: #f8f3ec;
  --champagne: #c8a96a;
  --charcoal: #2d2a26;
}
```

6.2 Mobile-first

Първо пишем стилове за телефон. После добавяме `@media (min-width: ...)` или Tailwind префикси.

```
className="px-5 sm:px-8 lg:px-12"
//      mobile  tablet  desktop
```

6.3 Breakpoints

Breakpoint	Пиксели	Употреба в проекта
default	0–639px	Мобилен layout

sm:	640px+	По-широк padding
-----	--------	------------------

lg:	1024px+	4 колони гестинации
-----	---------	---------------------

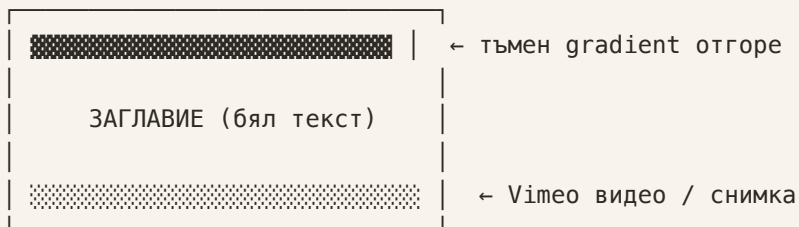
CSS @media	768px, 1280px	Hero, модали
------------	---------------	--------------

6.4 Gradients u overlays

Body gradient:

```
background:
  radial-gradient(circle at 12% 6%, rgba(200, 169, 106, 0.16), transparent 28rem),
  linear-gradient(180deg, var(--ivory), #fbf7f1 46%, var(--warm-beige));
```

Hero overlay — тъмен градиент върху видеото за четим текст:



6.5 Safe area (iPhone notch)

```
--safe-bottom: env(safe-area-inset-bottom, 0px);  
--mobile-sticky-bottom: calc(1rem + var(--safe-bottom));
```

Sticky call бутонът не се криве за home indicator на iPhone.

6.6 Animations

- `prefers-reduced-motion` — изключва видео и анимации за достъпност
- CSS transitions на hover за карти и бутони
- Hero: плавно скриване на poster снимката когато видеото тръгне

7. SEO

7.1 Защо SEO е важно?

Клиентите търсят „люксови пътувания Малдиви“, „круиз Средиземно море“. Без SEO сайтът няма да се появява в Google.

7.2 metadata (title, description)

```
// app/[locale]/layout.tsx  
export async function generateMetadata({ params }) {  
  return {  
    title: meta.title,  
    description: meta.description,  
    alternates: { canonical, languages: getAlternateLanguages() },  
  };  
}
```

- **title** — заглавие в таба и в Google резултати
- **description** — кратко описание под линка в Google

7.3 canonical

Казва на Google: „Това е официалният URL на страницата.“

```
canonical: localePath(locale, `/destinations/${destination.slug}`)  
// → https://barakovaluxurytravel.com/bg/destinations/maldives
```

Прегомвратява duplicate content между `/bg` и `/en` без hreflang.

7.4 sitemap.xml

`app/sitemap.ts` генерира списък с всички URL-и:

```
/bg  
/en  
/bg/destinations/maldives  
/en/destinations/maldives  
... (95 URL-а общо)
```

Google чете sitemap и индексира страниците по-бързо.

7.5 robots.txt

`app/robots.ts`:

```
User-agent: *  
Allow: /  
Disallow: /api/  
Sitemap: https://barakovaluxurytravel.com/sitemap.xml
```

API routes не трябва да се индексират.

7.6 Open Graph

Когато някой сподели линк във Facebook/Viber:

```
openGraph: {  
  title: "...",  
  description: "...",  
  images: [{ url: heroImage, width: 1024, height: 617 }],  
}
```

Показва се preview карта със снимка и текст.

7.7 Twitter Cards

```
twitter: {  
  card: "summary_large_image",  
  title: "...",  
  images: [heroImage],  
}
```

Същото за Twitter/X.

7.8 JSON-LD (Structured Data)

Машинно четим JSON в `<head>`:

```
{  
  "@type": "TravelAgency",  
  "name": "Barakova Luxury Travel",  
  "email": "info@barakovaluxurytravel.com"  
}
```

На детайлни страници:

- Дестинация → `TouristDestination`
- Круиз → `TouristTrip`
- Блог → `Article`

7.9 hreflang

Казва на Google: „Има българска и английска версия.“

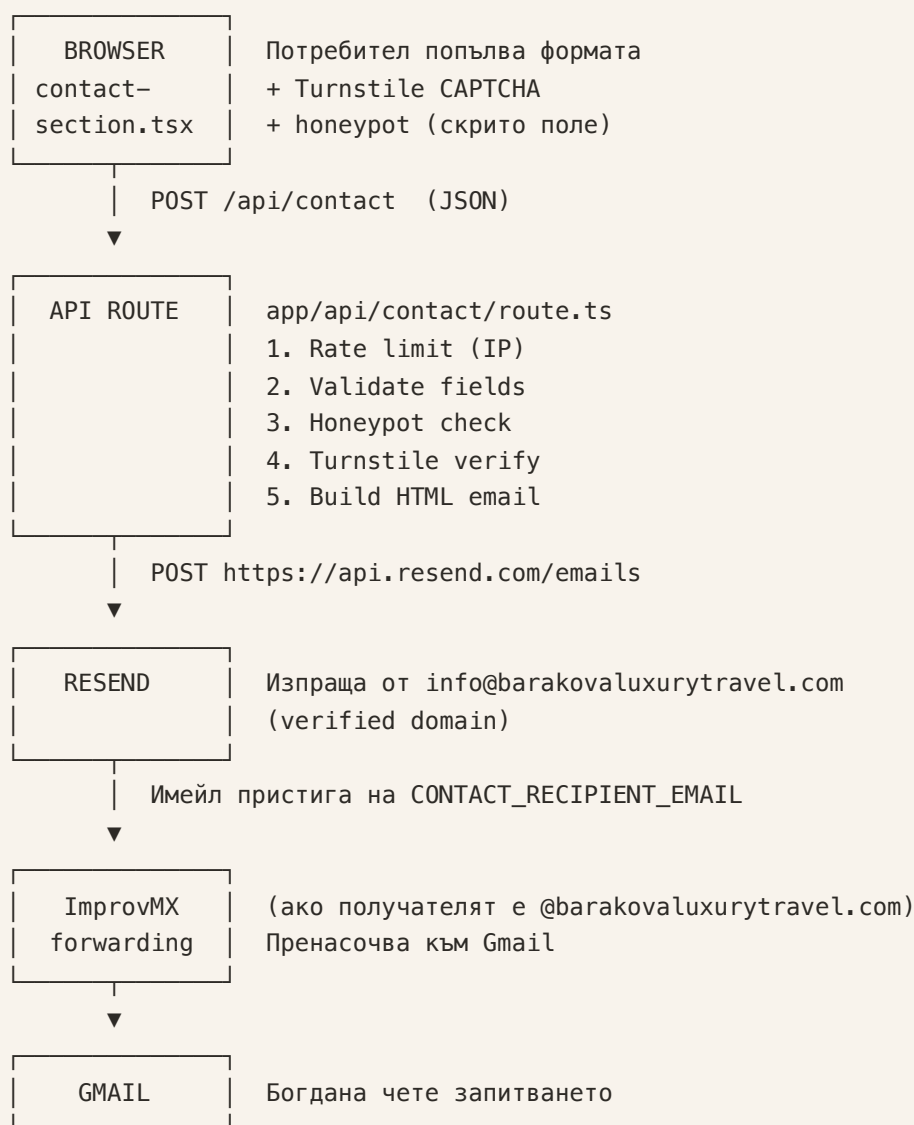
```
// constants/i18n.ts  
export function getAlternateLanguages(path = "") {  
  return {  
    bg: localePath("bg", path),  
    en: localePath("en", path),  
    "x-default": localePath("bg", path),  
  };  
}
```

Резултат в HTML:

```
<link rel="alternate" hreflang="bg" href="/bg/destinations/maldives" />
<link rel="alternate" hreflang="en" href="/en/destinations/maldives" />
```

8. Контактна форма

8.1 Пълен поток



8.2 Стъпка по стъпка

Стъпка 1 — Browser (клиент)

Файл: `app/components/home/contact-section.tsx`

- Потребителят попълва: име, email, телефон, опционални полета
- `validateForm()` проверява задължителните полета
- Ако Turnstile е конфигуриран → изисква token
- `fetch("/api/contact", { method: "POST", body: JSON.stringify(...)} )`

Стъпка 2 — API Route (сървър)

Файл: `app/api/contact/route.ts`

1. Взима IP адреса (`x-forwarded-for`)
2. `isContactRateLimited(ip)` — max 5 заявки/минута
3. `normalizeBody()` — trim, max length
4. `validateContactBody()` — required fields, email format
5. Honeypot: ако `website` поле е попълнено → връща `{ ok: true }` тихо (бом)
6. `verifyTurnstileToken()` — Cloudflare проверка
7. `buildEmailHtml()` — HTML таблица с данните
8. `fetch` към Resend API

Стъпка 3 — Resend

- Изпраща HTML имейл
- `from: RESEND_FROM_EMAIL` (`info@barakovaluxurytravel.com`)
- `to: CONTACT_RECIPIENT_EMAIL`
- `reply_to`: имейлът на потребителя (за директен отговор)

Стъпка 4 — ImprovMX (опционално)

Ако `CONTACT_RECIPIENT_EMAIL=info@barakovaluxurytravel.com`, ImprovMX пренасочва към Gmail.

Стъпка 5 — Gmail

Клиентът вижда запитването в inbox-а си.

8.3 Environment Variables за имейл

```
RESEND_API_KEY=re_xxxx          # Secret – само на сървъра  
CONTACT_RECIPIENT_EMAIL=info@barakovaluxurytravel.com  
RESEND_FROM_EMAIL=Barakova Luxury Travel <info@barakovaluxurytravel.com>
```

`NEXT_PUBLIC_*` = видими в браузъра.

Без `NEXT_PUBLIC` = само на сървъра (секретни).

9. Security

9.1 Cloudflare Turnstile

- **Клиент:** `TurnstileWidget` генерира token
- **Сървър:** `verifyTurnstileToken()` праща token + secret key към Cloudflare
- Ако secret липсва → CAPTCHA се пропуска (dev mode)

9.2 CSP (Content Security Policy)

Файл: `next.config.ts`

Казва на браузъра: „Зареждай скринтове **само** от тези домейни.“

```
script-src 'self' https://www.googletagmanager.com https://challenges.cloudflare.com  
img-src 'self' https://images.unsplash.com  
frame-src https://player.vimeo.com
```

Ако хакер инжектира `<script src="evil.com">` → браузърът го блокира.

9.3 HSTS

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Браузърът **винаги** използва HTTPS. Не може downgrade към HTTP.

9.4 Rate Limiting

```
// lib/rate-limit.ts
// Production: Upstash Redis (споделен между serverless instances)
// Fallback: in-memory Map (само за dev)
maxRequestsPerWindow = 5 per minute per IP
```

Защита срещу flood атаки на формата.

9.5 Honeypot

Скрито поле `website` — невидимо за хора, попълва се от ботове:

```
<div aria-hidden="true" className="honeypot-field">
  <input name="website" tabIndex={-1} />
</div>
```

Бот попълва → API връща fake success, не изпраща имейл.

9.6 Environment Variables

Променлива	Къде	Секретна?
RESEND_API_KEY	Server	✓ Да
TURNSTILE_SECRET_KEY	Server	✓ Да
UPSTASH_REDIS_REST_TOKEN	Server	✓ Да
NEXT_PUBLIC_TURNSTILE_SITE_KEY	Browser	✗ Публична
NEXT_PUBLIC_GA_ID	Browser	✗ Публична

Никогa не commit-вай `.env.local` в Git!

9.7 HTML Escaping

```
// lib/contact.ts
function escapeHtml(value: string) {
  return value.replaceAll("<", "&lt;").replaceAll(">", "&gt;")...
}
```

Предотвратява XSS в имейл HTML, ако някой пише `<script>` в формата.

10. Performance

10.1 next/image

```
<Image
  src={imageSrc}
  alt={alt}
  fill
  quality={60}
  sizes="(min-width: 1024px) 25vw, 50vw, 100vw"
/>
```

Next.js автоматично:

- Конвертира към WebP/AVIF
- Resize спорег `sizes`
- Lazy load (освен `priority` images)

10.2 Lazy loading на Vimeo

Hero видеото **не** се зарежда веднага:

```
// hero-background.tsx
// 1. Показва poster снимка веднага
// 2. След requestIdleCallback / timeout → зарежда iframe
// 3. Когато видеото тръгне → скрива poster-а
```

Защо? Видеото е тежко. Първото впечатление трябва да е бързо.

10.3 Mobile video optimization

```
if (isMobileViewport()) {  
  postToVimeoPlayer(iframe, "setQuality", "360p");  
}
```

На телефон → по-ниско качество → по-малко данни.

10.4 SSG (Static Site Generation)

`generateStaticParams` + `npm run build` → 95 pre-rendered HTML страници.

Потребителят получава готов HTML от CDN — не чака server rendering.

10.5 Bundle size

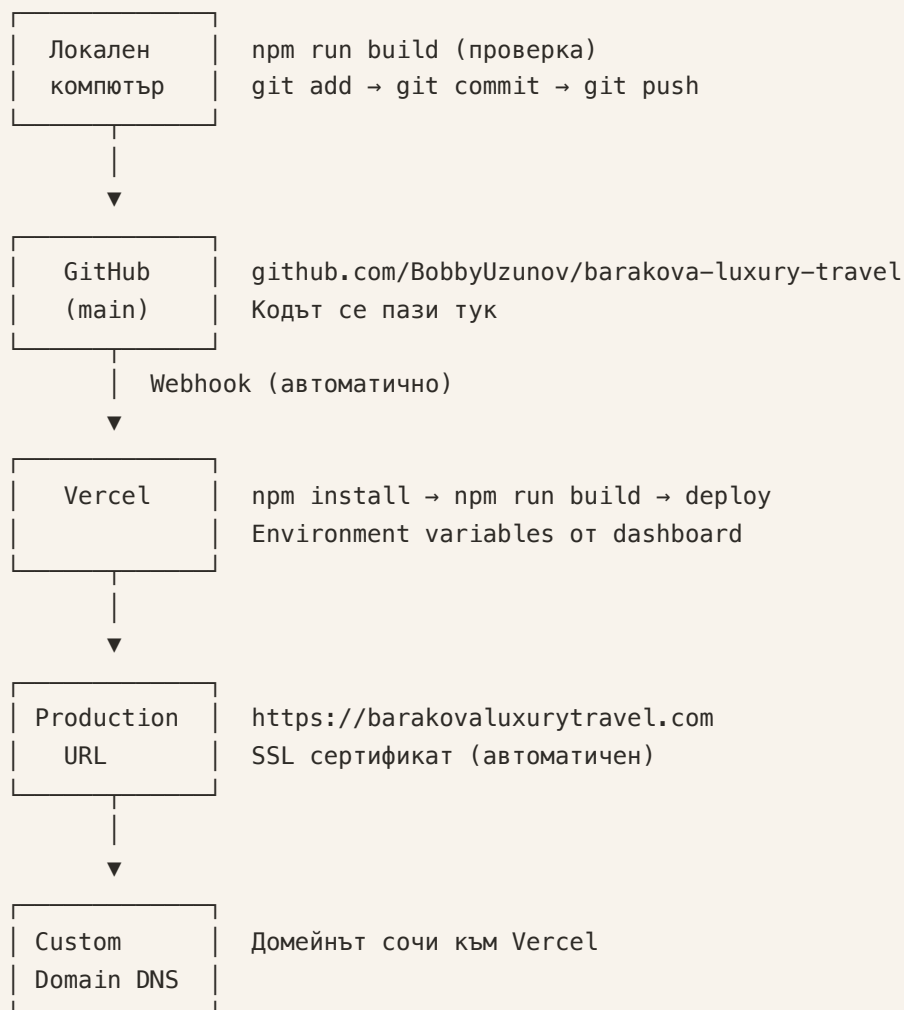
- Server Components не отиват в browser bundle
- Само `"use client"` компоненти добавят JavaScript
- `home-page.tsx` е най-големият client bundle (форми, модали, hero)

10.6 Image optimization scripts

```
npm run optimize-images # JPEG → WebP с sharp  
npm run generate-icons  # SVG → PNG за PWA
```

11. Deployment

11.1 Пълен deployment поток



11.2 DNS настройки (опростено)

barakovaluxurytravel.com

- A/CNAME record → Vercel servers (сайт)
- MX records → ImprovMX (имейл forwarding)
- TXT records → Resend domain verification (SPF, DKIM)

11.3 SSL

Vercel автоматично издава Let's Encrypt сертификати. HSTS header го enforce-ва.

11.4 Vercel Environment Variables

Задавам се в: Vercel Dashboard → Project → Settings → Environment Variables

Production и Preview могат да имат различни стойности.

12. Git

12.1 Основни команди

```
git status      # Какво е променено?  
git add .       # Stage всички промени  
git commit -m "... " # Записва snapshot  
git push        # Качва в GitHub → trigger deploy
```

12.2 Branch

```
main ← основен клон, production deploy
```

За по-големи проекти:

```
feature/new-blog → работа → Pull Request → merge в main
```

12.3 Типичен workflow

1. Променяш код локално
2. `npm run build` (проверка)
3. `git add + commit`
4. `git push`
5. Vercel deploy-ва автоматично (~1-2 мин)
6. Проверяваш live сайта

12.4 Какво НЕ трябва в Git

- `.env.local` (секрету)
- `node_modules/`
- `.next/` (build output)

Те са в `.gitignore`.

13. Реални примери

13.1 localePath — изграждане на URL

```
// constants/i18n.ts
export function localePath(locale: Locale, path = "") {
  if (!path || path === "/") {
    return `/${locale}`;          // → /bg
  }
  return `/${locale}${path.startsWith("/") ? path : `/${path}`}`;
  // localePath("en", "/privacy") → /en/privacy
}
```

Защо така? Един източник на истина за URL-и. Няма hardcoded `/bg/contact`.

13.2 Intercepted Modal — затваряне с `router.back()`

```
// intercepted-content-modal.tsx
export function InterceptedContentModal({ content, item, locale }) {
  const router = useRouter();

  return (
    <ContentModal
      item={item}
      onClose={() => router.back()}          // URL се връща на /bg
      onContact={() => router.push(localizedHash(locale, "#contact"))}
    />
  );
}
```

Защо `router.back()` ? URL вече е `/bg/destinations/maldives` . `Back` връща на `/bg` и модалът изчезва.

13.3 API Route — rate limit + validation

```
// app/api/contact/route.ts (опростено)
export async function POST(request: Request) {
  const ip = getClientIp(request);

  if (await isContactRateLimited(ip)) {
    return NextResponse.json({ message: "...", { status: 429 }});
  }

  const body = normalizeBody(await request.json());
  const validation = validateContactBody(body);

  if (!validation.ok) {
    if (validation.error === "honeypot") return NextResponse.json({ ok: true });
    return NextResponse.json({ message: "...", { status: 400 }});
  }

  // ... turnstile, resend ...
  return NextResponse.json({ ok: true });
}
```

Peg no peg:

1. IP → rate limit
2. Parse JSON → normalize
3. Validate → honeypot / fields / email
4. Turnstile
5. Resend
6. Success response

13.4 generateMetadata за гестинация

```
// destinations/[slug]/page.tsx
export async function generateMetadata({ params }) {
  const { locale, slug } = await params;
  const destination = findSeoDestination(locale, slug);

  return {
    title: `${destination.name} – луксозно пътуване | Barakova Luxury Travel`,
    description: destination.detail,
    alternates: {
      canonical: localePath(locale, `/destinations/${slug}`),
      languages: getAlternateLanguages(`/destinations/${slug}`),
    },
  };
}
```

Всяка гестинация има **уникален title** в Google.

13.5 Turnstile Widget

```
// turnstile-widget.tsx
const siteKey = process.env.NEXT_PUBLIC_TURNSTILE_SITE_KEY;

if (!siteKey) return null; // Dev без keys → няма CAPTCHA

return (
  <Turnstile
    siteKey={siteKey}
    onSuccess={onTokenChange} // token → state → POST body
    onExpire={() => onTokenChange("")}
  />
);
```

13.6 Hero Video — lazy load

```
// hero-background.tsx (логика)
useEffect(() => {
  if (!canUseHeroVideo()) return; // reduced motion / data saver

  // Preconnect към Vimeo
  // След delay → setLoadVideo(true) → рендерира iframe
  // Слуша postMessage "playing" → setVideoPlaying(true)
}, []);
```

13.7 Sitemap generation

```
// app/sitemap.ts
for (const locale of locales) { // bg, en
  for (const slug of destinationSlugs) { // maldives, seychelles...
    entries.push({
      url: `${siteUrl}${localePath(locale, `/destinations/${slug}`)}`,
      priority: 0.8,
    });
  }
}
```

12 дестинации × 2 езика = 24 URL-а само за дестинации.

14. Какво научих

След този проект вече разбираш:

Frontend

- HTML семантика (header, main, section, article)
- CSS: variables, gradients, responsive, mobile-first
- Tailwind utility classes
- React: components, props, state, effects
- TypeScript: types, interfaces, type guards

Next.js

- App Router file structure
- Server vs Client Components
- Layouts u nested routing
- Dynamic routes `[slug]`
- Intercepting routes `@modal`
- `generateStaticParams` u SSG
- `generateMetadata` за SEO
- API Routes
- `next/image` optimization
- `proxy.ts` redirects

Backend basics

- REST API (POST `/api/contact`)
- Request validation
- Rate limiting
- External API integration (Resend, Turnstile)
- Environment variables

DevOps

- Git: commit, push, branch
- GitHub + Vercel CI/CD
- DNS, SSL, custom domain
- Environment variables в production

Security & Compliance

- CAPTCHA, honeypot, rate limiting
- CSP, HSTS headers
- GDPR cookie consent
- HTML escaping (XSS prevention)

SEO

- Metadata, canonical, hreflang

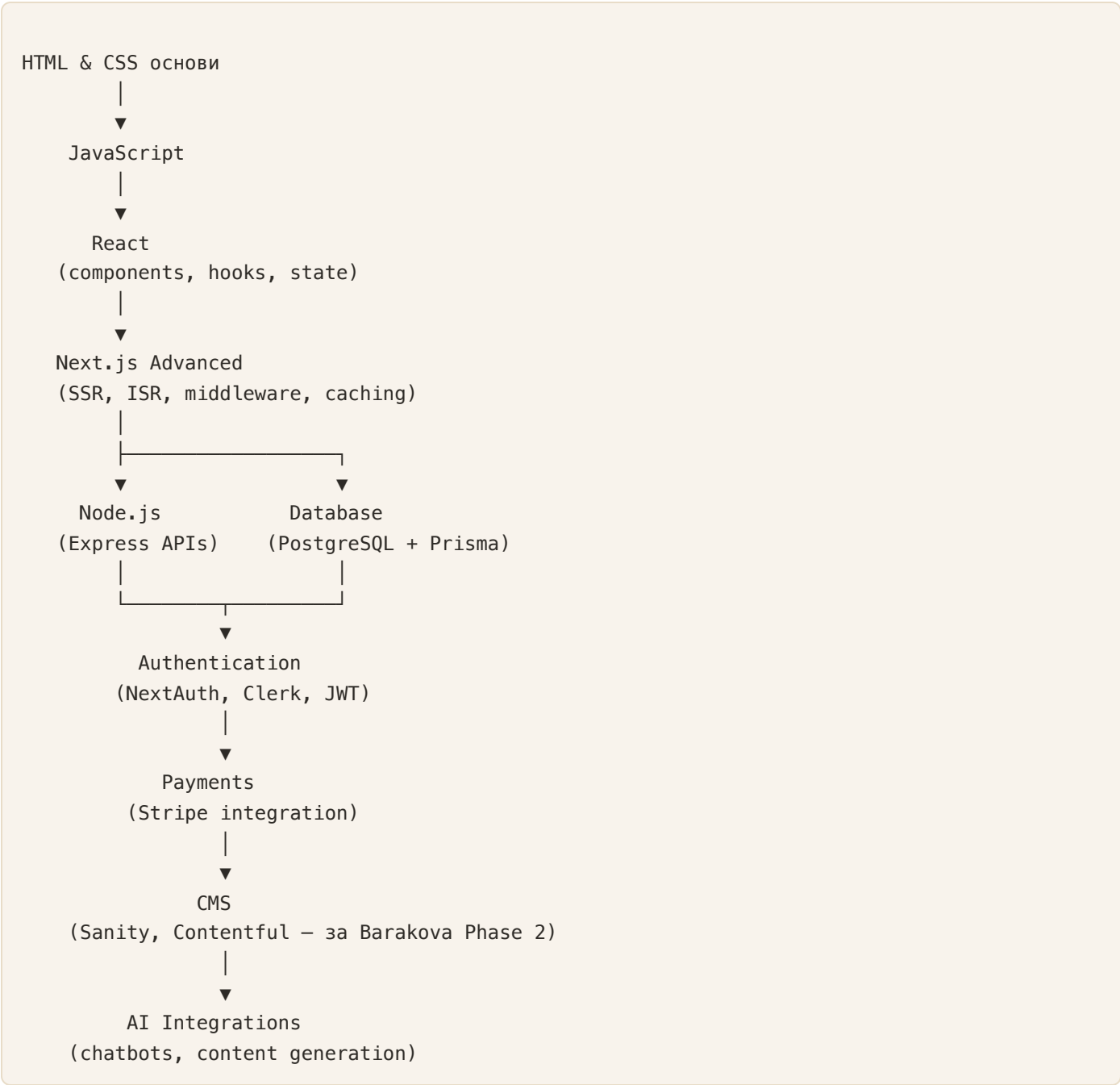
- Sitemap, robots.txt
- Open Graph, Twitter Cards
- JSON-LD structured data

Real-world skills

- i18n (г8уезичен сайт)
 - Contact form end-to-end
 - Email delivery chain
 - Performance optimization
 - Accessibility (focus trap, aria labels, reduced motion)
-

15. Какво да уча след това

Roadmap



Препоръчан ред за учене

#	Тема	Защо
1	JavaScript fundamentals	Основа на всичко

2	React hooks (useState, useEffect)	Вече ги ползваш — задълбочи
3	TypeScript	Вече го ползваш — научи generics, utility types
4	Next.js docs (App Router)	Официалната документация
5	CSS Grid & Flexbox	Layout без framework
6	Node.js basics	Разбиране на API routes
7	SQL + Prisma	Когато трябва база данни
8	Testing (Jest, Playwright)	<code>contact.test.ts</code> е начало
9	Accessibility (WCAG)	a11y в модала, форми
10	Sanity CMS	Следваща фаза на Barakova

Практически упражнения с този проект

1. Добави нова дестинация в `content-bg.ts` + slug в `seo-content.ts`
2. Смени hero цветовете в `globals.css` :root
3. Добави нов blog post
4. Напиши Playwright тест за контакт формата
5. Добави `/bg/faq` страница от нулата

Полезни ресурси

- [Next.js Learn](#) — безплатен курс
- [React docs](#) — официална документация
- [TypeScript Handbook](#)
- [web.dev](#) — performance и SEO
- [MDN Web Docs](#) — HTML, CSS, JS референция

Индекс

Термин	Секция
API Route	4, 8, 13
App Router	2, 4
CAPTCHA / Turnstile	2, 9, 13
Client Component	2, 5
CMS	1, 15
Composition	5
Contact Form	8
CSP	9
Deployment	11
DNS	11
Dynamic Routes	4
Environment Variables	8, 9, 11
generateMetadata	4, 7, 13
generateStaticParams	4, 10
Git	12
GitHub	11, 12
globals.css	6

Honeypot	9
hreflang	7
HSTS	9
i18n	3, 4, 7
ImprovMX	2, 8
Intercepting Routes	4, 13
JSON-LD	2, 7
layout.tsx	4
Lazy Loading	10
Metadata	4, 7
Middleware / proxy.ts	4
Mobile-first	6
next/image	10
Open Graph	7
Parallel Routes	4
Performance / SSG	10
Props	5
Rate Limiting	9
React	2, 5

Resend	2, 8
robots.txt	7
Routing	4
Security	9
Server Component	2, 5
sitemap.xml	7
SSL	11
State	5
Structured Data	2, 7
Tailwind CSS	2, 6
TypeScript	2
Vercel	2, 11
Vimeo	2, 10
XSS	9

Край на документа. Този учебник описва състоянието на проекта към юли 2026. При промени в кода, актуализирай съответните секции.